

UNIVERSIDAD DE SANTANDER — UDES

Facultad de Ciencias e Ingeniería

INFORME TÉCNICO DE PROYECTO

Libros API — Despliegue en Railway

Asignatura: Programación Web

Unidad: 12 — Post 1

Estudiante: Andrés Toscano

Universidad: UDES — Semestre 7

Fecha: 19 de mayo de 2026

1. URL de la Aplicación Desplegada

La API fue desplegada exitosamente en Railway y se encuentra disponible en la siguiente URL pública:

```
https://toscano-post1-u12-production.up.railway.app
```

Esta URL fue generada desde el panel de Railway en Settings → Networking → Generate Domain, y da acceso a todos los endpoints de la API en producción, conectados a la base de datos PostgreSQL 16 provisionada en la misma plataforma.

Ejemplo de acceso rápido para verificar disponibilidad:

```
curl https://toscano-post1-u12-production.up.railway.app/actuator/health  
  
// Respuesta esperada:  
{ "status": "UP" }
```

2. Capturas de Pantalla

A continuación se describen las capturas de pantalla que documentan el despliegue y funcionamiento de la API. Estas corresponden a los archivos ubicados en la carpeta capturas/ del proyecto.

2.1 Panel de Railway

Captura: PanelRailway.png

El panel de Railway muestra el proyecto con dos servicios activos: el servicio de la aplicación Spring Boot y el servicio de base de datos PostgreSQL. En él se pueden observar:

- Estado de despliegue: activo (verde) para ambos servicios.
- El servicio de la app construido a partir del repositorio GitHub vinculado.
- La URL de producción generada bajo el dominio .up.railway.app.
- Las variables de entorno configuradas en el servicio de la aplicación.
- Historial de despliegues con el hash del commit de GitHub.

[Captura: PanelRailway.png]

2.2 Health Check — /actuator/health

Captura: URL-Ok.png

El endpoint de Spring Boot Actuator confirma que la aplicación se encuentra en estado UP y conectada correctamente a la base de datos PostgreSQL. La respuesta del healthcheck fue:

```
GET /actuator/health

HTTP/1.1 200 OK
Content-Type: application/json

{"status":"UP"}
```

[Captura: URL-Ok.png]

2.3 Endpoint — GET /api/libros (todos los libros)

Captura: ListarLibros-Ok.png

El endpoint GET /api/libros retorna el listado completo de los 5 libros semilla insertados por data.sql al iniciar la aplicación. La respuesta confirma que la persistencia en PostgreSQL funciona correctamente en el ambiente de producción:

```
GET /api/libros

HTTP/1.1 200 OK
[
  { "id":1, "titulo":"Cien años de soledad", "autor":"Gabriel García Márquez",
    "anio":1967, "genero":"Realismo mágico", "disponible":true },
  { "id":2, "titulo":"El señor de los anillos", "autor":"J.R.R. Tolkien",
    "anio":1954, "genero":"Fantasía", "disponible":true },
  { "id":3, "titulo":"Clean Code", "autor":"Robert C. Martin",
    "anio":2008, "genero":"Tecnología", "disponible":true },
  { "id":4, "titulo":"1984", "autor":"George Orwell",
    "anio":1949, "genero":"Distopía", "disponible":false },
  { "id":5, "titulo":"El Quijote", "autor":"Miguel de Cervantes",
    "anio":1605, "genero":"Clásico", "disponible":true }
]
```

[Captura: ListarLibros-Ok.png]

2.4 Endpoint — GET /api/libros/disponibles

Captura: LibrosDisponibles-Ok.png

El endpoint GET /api/libros/disponibles utiliza la consulta derivada findByDisponibleTrue() del repositorio JPA y retorna únicamente los libros con el campo disponible: true. De los 5 libros semilla, 4 cumplen esta condición (el libro "1984" de George Orwell tiene disponible: false).

[Captura: LibrosDisponibles-Ok.png]

2.5 Endpoint — GET /api/libros (vista en navegador)

Captura: ListarLibros.png

Vista complementaria del endpoint de listado general accedido directamente desde el navegador web, confirmando que la URL pública de Railway responde correctamente desde un cliente HTTP estándar sin necesidad de herramientas adicionales.

[Captura: ListarLibros.png]

3. Configuración de Variables de Entorno en Railway

Uno de los aspectos más importantes del despliegue en Railway es la correcta configuración de las variables de entorno que permiten a la aplicación Spring Boot conectarse a la base de datos PostgreSQL provisionada en la nube y activar el perfil de producción adecuado.

3.1 Pasos de Configuración

El proceso de configuración de variables de entorno en Railway se realizó mediante los siguientes pasos:

Paso 1 — Crear el proyecto en Railway

Se creó un nuevo proyecto en railway.app y se vinculó al repositorio de GitHub que contiene el código fuente de la API. Railway detecta automáticamente el Dockerfile y lo usa para construir la imagen de producción.

Paso 2 — Aprovisionar PostgreSQL

Dentro del mismo proyecto de Railway se añadió un servicio de base de datos PostgreSQL 16 usando la opción Add Service → Database → PostgreSQL. Railway provisiona automáticamente la base de datos y genera las variables de conexión internas del proyecto.

Paso 3 — Configurar las variables en el servicio de la aplicación

Desde el panel del servicio de la aplicación (pestaña Variables), se configuraron las siguientes variables de entorno. Railway permite referenciar variables de otros servicios del mismo proyecto usando la sintaxis `${NombreServicio.VARIABLE}`:

Variable	Valor configurado
<code>SPRING_PROFILES_ACTIVE</code>	prod
<code>DATABASE_URL</code>	<code>\${Postgres.DATABASE_URL}</code>
<code>DB_USER</code>	<code>\${Postgres.PGUSER}</code>
<code>DB_PASS</code>	<code>\${Postgres.PGPASSWORD}</code>

3.2 Descripción de Cada Variable

SPRING_PROFILES_ACTIVE = prod

Activa el perfil de Spring Boot definido en `application-prod.properties`, que configura el driver de PostgreSQL, deshabilita la consola H2 y ajusta los niveles de logging para producción. Sin esta variable, la aplicación arrancaría con el perfil por defecto (H2 en memoria), lo que causaría un fallo al intentar conectarse a PostgreSQL.

DATABASE_URL — URL de conexión JDBC

La URL JDBC completa hacia la base de datos PostgreSQL provisionada por Railway. Utiliza la referencia interna `${Postgres.DATABASE_URL}` para resolver dinámicamente el host, puerto y nombre de la base de datos del servicio PostgreSQL del mismo proyecto. En `application-prod.properties` es referenciada como `${DATABASE_URL}`.

DB_USER — Usuario de la base de datos

El usuario con permisos sobre la base de datos `librosdb`. Referencia la variable `PGUSER` del servicio PostgreSQL de Railway. Spring Boot la inyecta en la configuración del `DataSource` como `spring.datasource.username`.

DB_PASS — Contraseña de la base de datos

La contraseña correspondiente al usuario de la base de datos. Referencia la variable `PGPASSWORD` del servicio PostgreSQL de Railway. Al usar referencias internas de Railway (`${...}`), el valor real nunca queda expuesto en el código fuente ni en el repositorio de GitHub, lo que constituye una buena práctica de seguridad.

3.3 Cómo se Consumen las Variables en la Aplicación

En el archivo `application-prod.properties`, las variables de entorno se consumen mediante la sintaxis estándar de Spring Boot `${NOMBRE_VARIABLE}`:

```
# application-prod.properties
```

```
spring.datasource.url=${DATABASE_URL}
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASS}
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
```

De esta forma, la aplicación nunca tiene credenciales hardcodeadas: en desarrollo local se usa H2 en memoria (sin credenciales), y en producción Railway inyecta los valores reales en tiempo de ejecución mediante las variables de entorno configuradas en el panel.

4. Endpoints Verificados en Producción

La siguiente tabla resume todos los endpoints expuestos por la API, verificados contra el ambiente de producción en Railway:

Método	Ruta	Descripción	Código HTTP
GET	/api/libros	Listar todos los libros	200 OK
GET	/api/libros/{id}	Obtener libro por ID	200 / 404
GET	/api/libros/disponibles	Listar libros disponibles	200 OK
GET	/api/libros/genero/{genero}	Filtrar por género	200 OK
GET	/api/libros/autor?nombre=X	Buscar por autor	200 OK
POST	/api/libros	Crear un libro	201 Created
PUT	/api/libros/{id}	Actualizar un libro	200 / 404
DELETE	/api/libros/{id}	Eliminar un libro	204 No Content
GET	/actuator/health	Estado de salud de la app	200 OK

Todos los endpoints fueron verificados mediante peticiones HTTP directas a la URL de producción, confirmando el correcto funcionamiento del ciclo completo: Docker multi-stage build → Railway CI/CD → PostgreSQL en la nube.

5. Conclusiones

- La API Libros fue desplegada exitosamente en Railway con PostgreSQL 16 como motor de base de datos en producción, confirmado mediante el endpoint /actuator/health que retorna {"status":"UP"}.
- La configuración de variables de entorno en Railway permite separar por completo las credenciales del código fuente, siguiendo la metodología 12-Factor App para aplicaciones cloud-native.

- El Dockerfile multi-stage produce una imagen final por debajo de los 300 MB usando solo el JRE de Alpine, con un usuario no root (spring) como buena práctica de seguridad.
- El uso de perfiles de Spring Boot (dev / prod) facilita el desarrollo local con H2 en memoria y el despliegue en producción con PostgreSQL sin necesidad de modificar el código.
- La arquitectura en capas (Controller → Service → Repository) con manejo centralizado de excepciones mediante `@RestControllerAdvice` garantiza respuestas HTTP semánticamente correctas (200, 201, 204, 400, 404) en todos los escenarios.